



Princeton Computer Science Contest – Fall 2025

Problem 6: Tetris Melons ($10 \times 2 = 20$ points)

By Joshua Lin

Introduction

You're trying to become the cheapest watermelon grower on the East Coast. Unfortunately, your melons have very strange shapes: each one looks like a small Tetris-like piece cut from a 5×5 grid. To cut costs, you want to pack your melons into as few trucks as possible. Each truck has a 10×10 loading floor made of unit grid cells. Melons cannot overlap or be cut, but you may rotate them. Besides minimizing the number of trucks, you also care about how efficiently the space inside each truck is used. Packings that leave small, isolated holes of unused space are considered worse than compact packings. Given a multiset of melon shapes, pack them into 10×10 trucks using as few trucks as possible, and among packings with the same number of trucks, make the packing as compact as possible.

Note: this problem might look intimidating, but there's a lot of starter code provided for you!

Formal Model

Each truck has a loading floor that is a 10×10 grid of unit cells with integer coordinates

$$\{0, 1, \dots, 9\} \times \{0, 1, \dots, 9\}.$$

You are given N melons ("melons"). Each melon i is described by a 5×5 bitmap of occupied cells.

Melon shapes

For melon i ($0 \leq i < N$):

- It is described by a 5×5 grid of characters '.' (empty) and '#' (occupied).
- The rows are given from bottom to top (row 0 is the bottom row, row 4 is the top row).
- The columns in each row are given from left to right (column 0 is the leftmost).
- At least one cell is '#', and at most 10 cells are '#'.

Princeton Computer Science Contest – Fall 2025



CITADEL | CITADEL Securities



PRINCETON
Computer Science



Department of
Mathematics



Princeton Computer Science Contest – Fall 2025

In local coordinates, we can view melon i as a set of occupied cells

$$\mathcal{S}_i = \{(u, v) \mid 0 \leq u, v \leq 4, \text{ cell at column } u, \text{ row } v \text{ is } '\#'\}.$$

You may rotate each melon i by $90^\circ \cdot r_i$ degrees clockwise, where $r_i \in \{0, 1, 2, 3\}$. Let $\mathcal{S}_i^{(r_i)}$ denote the set of occupied local cells after applying the chosen rotation r_i to melon i .

Placements

You may use K trucks, numbered $0, 1, \dots, K - 1$. A placement chooses for each melon i :

- an integer truck index $t_i \in \{0, \dots, K - 1\}$,
- an integer bottom-left coordinate (x_i, y_i) with $0 \leq x_i, y_i \leq 9$,
- a rotation $r_i \in \{0, 1, 2, 3\}$.

For melon i with placement (t_i, x_i, y_i, r_i) , each occupied local cell $(u, v) \in \mathcal{S}_i^{(r_i)}$ is mapped to a truck cell

$$(x_i + u, y_i + v)$$

in truck t_i . A placement is *valid* if and only if:

- For every melon i and every $(u, v) \in \mathcal{S}_i^{(r_i)}$,

$$0 \leq x_i + u \leq 9, \quad 0 \leq y_i + v \leq 9,$$

i.e., all occupied cells lie inside the 10×10 grid of the chosen truck.

- For any two distinct melons in the same truck, none of their *occupied* ('#') truck cells coincide. Melons may touch at edges or corners.
- $K = \max_i t_i + 1$ (you must use trucks $0, \dots, K - 1$ without gaps).

Princeton Computer Science Contest – Fall 2025



CITADEL | CITADEL Securities



PRINCETON
Computer Science



Department of
Mathematics



Princeton Computer Science Contest – Fall 2025

Input

The input describes one test case.

```
N
shape[0] row 0
shape[0] row 1
shape[0] row 2
shape[0] row 3
shape[0] row 4
shape[1] row 0
...
shape[N-1] row 4
```

Assumptions:

- Each row is a string of exactly 5 characters, each either ‘.’ or ‘#’.
- For each melon, rows are given from bottom to top: row 0 is the bottom row, row 4 is the top row.
- Each melon has at least one ‘#’ and at most 10 ‘#’.

Output

You must print:

- An integer K : the number of trucks you are using.
- Then N lines, one per melon (in the same order as input), each of the form:

```
ti xi yi ri
```

where each value is as defined previously.

Princeton Computer Science Contest – Fall 2025



CITADEL | CITADEL Securities



PRINCETON
Computer Science



Department of
Mathematics



Princeton Computer Science Contest – Fall 2025

Starter code is provided [here](#) in both Python and Java. `melons.txt` is the input file; `helpers` provides some utility functions such as reading input; `simple` implements the strategy of “place every watermelon on its own separate truck”; `score` is the program which will evaluate your submission. Example instructions for how to run the programs are located in the comments of the `simple` files.

Part A (10 Points)

Find a solution for the given input file with $K \leq 12$, where K is the number of trucks. Submit [here](#).

Part B (10 Points)

It’s bad for watermelons to be rolling all over the place while the trucks are moving. Thankfully, if the watermelons are placed compactly enough, they can be tied down and secured. This part explores solutions which minimize one particular choice of metric for such compactness.

Holes Penalty

Fix one truck t . Consider its 10×10 grid of cells. Each cell is either:

- *occupied* (part of at least one melon), or
- *empty* (not occupied by any melon).

We consider 4-neighbor adjacency (up/down/left/right).

- An *empty region* is a maximal connected component of empty cells under 4-neighbor adjacency.
- An empty region is a *border region* if it contains at least one cell on the boundary of the truck (that is, some cell with $x = 0$, $x = 9$, $y = 0$, or $y = 9$).
- An empty region that is not a border region is called a *hole*.

For each hole h , let $|h|$ be its number of cells. The *hole penalty* for truck t is

$$H_t = \sum_{\text{holes } h} |h|^2.$$

Princeton Computer Science Contest – Fall 2025



CITADEL | CITADEL Securities



PRINCETON
Computer Science



Department of
Mathematics



Princeton Computer Science Contest – Fall 2025

Fragmentation Penalty

Again for a fixed truck t , consider only the occupied cells. Using 4-neighbor adjacency, count the number of connected components of occupied cells; call this number comp_t . The *fragmentation penalty* is

$$F_t = \max(0, \text{comp}_t - 1).$$

This is zero if all melons in the truck form a single connected blob, and increases if they are scattered into multiple disconnected clusters. For the precise calculation, see `compute_truck_penalty` in `score.py`.

Compactness Penalty

For each truck t , define its total penalty by

$$C_t = H_t + 5 \cdot F_t,$$

The global compactness penalty over all trucks is

$$C = \sum_{t=0}^{K-1} C_t.$$

Objective

Submit a solution with $K = 12, C \leq 70$ for 3 points. Submit a solution with $K = 12, C \leq 45$ for an additional 7 points (i.e., the full 10 points). Submit [here](#).

Alternatively, you may submit a solution with $K \leq 11$ for the full 10 points, no matter your value of C . (Note from problem writer: I don't think this is possible.)

Princeton Computer Science Contest – Fall 2025



CITADEL | CITADEL Securities



PRINCETON
Computer Science



Department of
Mathematics



Princeton Computer Science Contest – Fall 2025

Examples

Example Input

```
2
##...
##...
.....
.....
.....
##...
#....
.....
.....
.....
```

This describes $N = 2$ melons. Melon 0 is a 2×2 square; melon 1 is an L-shaped piece of size 3.

Example Output

One valid output is

```
1
0 0 0 0
0 2 0 0
```

Explanation: We use one truck ($K = 1$).

- Melon 0 is placed in truck 0 at position $(x_0, y_0) = (0, 0)$ with rotation $r_0 = 0$. Its 2×2 block of occupied cells fills truck cells $(0, 0)$, $(1, 0)$, $(0, 1)$, $(1, 1)$.
- Melon 1 is placed in truck 0 at position $(x_1, y_1) = (2, 0)$ with rotation $r_1 = 0$. Its L-shape occupies truck cells $(2, 0)$, $(3, 0)$, $(2, 1)$.

There are no overlaps, and all occupied cells lie inside the 10×10 grid.

Princeton Computer Science Contest – Fall 2025



CITADEL | CITADEL Securities



PRINCETON
Computer Science



Department of
Mathematics